

Working with LLMs

Understand, translate, and validate SAS workflows

For data science professionals who know SAS
and are new to LLM collaboration.

**Teach a collaboration workflow,
not a collection of magic prompts.**

Concept and sample learning aids for team review. Not an approved rollout.

4

brief sessions

Python first

Model-neutral
Evidence-led

Reduce open choices. Make checks reproducible.

**We are reducing unnecessary choices and building reproducible checks.
We are not promising identical model responses.**

01

Understand

Review what the SAS program does.

02

Specify

State the behavior and acceptance rules.

03

Translate

Implement one agreed unit in Python.

04

Check

Run reviewed code against expectations.

05

Improve

Refactor only from a checked baseline.

A different implementation can pass. A repeated implementation can still be wrong.

The acceptance decision belongs to the analyst and is limited to the stated scope.

Four brief sessions, four visible outcomes

Approximately 25 minutes each. Access and basic environment setup happen before class.

SESSION	WHAT THE LEARNER PRODUCES
01 Read before you rewrite	A reviewed explanation and an unresolved-dependency list.
02 Ask for the behavior you need	A bounded MEAN translation request and candidate code.
03 Check more than “it runs”	An evidence record using SUBSTR or LENGTH checks.
04 Work through a small pipeline	A decomposed workflow with boundary and final-output checks.

5 min explain + 8 min demonstrate + 8 min practice + 4 min discuss

Make the consequential choices inspectable

MEAN is the first worked example. The guide contains the complete reusable request.

THE FAMILIAR SAS CALCULATION

```
average = mean(x, y, z);
```

1	2	3	Three rows.
4	.	6	One calculation.
.	.	.	Different missingness.

PRACTICAL HEADINGS, NOT AN ACRONYM

Task	One transformation.
Source and context	Source, schema, dependencies.
Environment	Approved Python and libraries.
Required behavior	Missingness, order, types, outputs.
Deliverable	Code and a requirement mapping.
Uncertainty and checks	Flag gaps; label evidence honestly.

HOW

Name the behavior to preserve.

WHY

The choice becomes reviewable.

WHY NOT

A fluent answer is not a check.

Check the result, not the prose

Use fixed expectations and a reviewed implementation. Keep ordinary, edge, and preservation checks distinct.

INPUT x, y, z	EXPECTED
1, 2, 3	2.0
4, missing, 6	5.0
all missing	missing
0, missing, missing	0.0
missing, -6, missing	-6.0

T1 / KNOWN ANSWERS
Original values and missing positions.

T2 / DISCRIMINATING CASES
Zero and a lone negative value.

T3 / PRESERVATION
Input unchanged; output structure retained.

Three named tests do not prove universal equivalence.

Python reference: locally checked | SAS baseline: pending | Live model trial: pending

A compact packet, with an executable companion

One set of requirements and examples, presented in formats people can use.

4-PAGE FIELD GUIDE

The workflow, practical requests, checks, and repairs.

2-SIDED DESK REFERENCE

Prompt headings on the front; evidence and repair on the back.

EXISTING WORKED MODULES

Use selected examples and appendices as needed.

MEAN NOTEBOOK PROTOTYPE

01	SAS source and required behavior
02	Prompt and saved-response record
03	Reviewed Python candidate
04	Executed checks and observed results
05	Evidence, limits, and acceptance decision

Static HTML preview included. No model API connection is required for the supplied demonstration.

Keep the pilot small enough to teach

Add languages and infrastructure only when they serve a defined learning objective.

CORE FOR REVIEW

Python as the first target.

Model-neutral collaboration.

Synthetic, bounded examples.

Evidence-led acceptance.

OPTIONAL DELIVERY

Jupyter as a companion.

Portal plus reviewed execution.

VS Code and SSH access.

Setup handled before class.

LATER EXTENSIONS

DuckDB for analytical SQL.

R, Stata, and Julia tracks.

Macros and stateful DATA steps.

Performance and redesign.

Analyst: requirements and acceptance. Model: proposed work. Runtime: observed execution.

Decide the pilot, not an entire AI curriculum

Use these materials to review scope, pacing, prerequisites, and evidence expectations.

AUDIENCE AND PACE	Are four short sessions realistic for these learners? Which existing examples should anchor the pilot?
ENVIRONMENT AND SUPPORT	What approved portal and execution path will be used? Who will confirm access and resolve semantic questions?
ACCEPTANCE AND FEEDBACK	What must learners demonstrate before completion? What observations will drive changes after the pilot?

The learner can direct the work, inspect the evidence,
and recognize when the translation is not ready to trust.